EXAMPLES

LLMProvider Module - Examples

Module: `digital.vasic.llmprovider` **Last Updated:** March 2026

Implementing a New Provider

Step 1: Define the Provider Struct

```go
package myprovider

import (
    "context"
    "fmt"
    "net/http"

    "digital.vasic.llmprovider"
    "digital.vasic.models"
)

type MyProvider struct {
    apiKey  string
    baseURL string
    client  *http.Client
    model   string
}

func NewMyProvider(apiKey, baseURL, model string) *MyProvider {
    return &MyProvider{
        apiKey:  apiKey,
        baseURL: baseURL,
        client:  &http.Client{Timeout: 60 * time.Second},
        model:   model,
    }
}
```

Step 2: Implement the LLMProvider Interface

```
func (p *MyProvider) Complete(ctx context.Context, req
        *models.LLMRequest) (*models.LLMResponse, error) {
    // Build the API request
    body := map[string]interface{}{
        "model":       p.model,
        "prompt":      req.Prompt,
        "max_tokens":  req.ModelParams.MaxTokens,
        "temperature": req.ModelParams.Temperature,
    }

    // Make the API call (simplified)
    resp, err := p.callAPI(ctx, "/v1/completions", body)
    if err != nil {
        return nil, fmt.Errorf("my-provider completion failed:
        %w", err)
    }

    return &models.LLMResponse{
        ID:         resp.ID,
        RequestID:  req.ID,
        ProviderID: "my-provider",
        Content:    resp.Text,
    }, nil
}

func (p *MyProvider) CompleteStream(ctx context.Context, req
        *models.LLMRequest) (<-chan *models.LLMResponse, error) {
    ch := make(chan *models.LLMResponse, 10)

    go func() {
        defer close(ch)

        // Stream responses from the API
        stream, err := p.callStreamAPI(ctx, "/v1/completions",
        req)
        if err != nil {
            return
        }

        for chunk := range stream {
            select {
            case ch <- &models.LLMResponse{
                ID:         chunk.ID,
                RequestID:  req.ID,
                ProviderID: "my-provider",
                Content:    chunk.Text,
            }:
            case <-ctx.Done():
                return
            }
```

```
        }
    }()

    return ch, nil
}

func (p *MyProvider) HealthCheck() error {
    // Check if the API is reachable
    resp, err := p.client.Get(p.baseURL + "/health")
    if err != nil {
        return fmt.Errorf("health check failed: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("unhealthy: HTTP %d", resp.StatusCode)
    }
    return nil
}

func (p *MyProvider) GetCapabilities()
        *models.ProviderCapabilities {
    return &models.ProviderCapabilities{
        Name:           "my-provider",
        SupportsStream: true,
        SupportsTools:  false,
        SupportsVision: false,
        MaxConcurrency: 10,
        MaxTokens:      4096,
    }
}

func (p *MyProvider) ValidateConfig(config
        map[string]interface{}) (bool, []string) {
    var errors []string

    if _, ok := config["api_key"]; !ok {
        errors = append(errors, "api_key is required")
    }
    if _, ok := config["base_url"]; !ok {
        errors = append(errors, "base_url is required")
    }

    return len(errors) == 0, errors
}
```

Using the Circuit Breaker

Basic Usage

```
import "digital.vasic.llmprovider"

// Create a provider
provider := myprovider.NewMyProvider(apiKey, baseURL, model)

// Wrap with a circuit breaker (default config: 5 failures, 30s
        timeout)
cb := llmprovider.NewDefaultCircuitBreaker("my-provider",
        provider)

// Use the circuit breaker as if it were a regular LLMProvider
resp, err := cb.Complete(ctx, req)
if err != nil {
    if errors.Is(err, llmprovider.ErrCircuitOpen) {
        log.Warn("Provider circuit is open, try another provider")
    }
    return nil, err
}
```

Custom Configuration

```
config := llmprovider.CircuitBreakerConfig{
    FailureThreshold:    3,                // Open after 3
        consecutive failures
    SuccessThreshold:    1,                // Close after 1
        success in half-open
    Timeout:             15 * time.Second, // Stay open for 15
        seconds
    HalfOpenMaxRequests:
        1,                 // Allow 1 test request in half-open
}

cb := llmprovider.NewCircuitBreaker("my-provider", provider,
        config)
```

Listening for State Changes

```
cb := llmprovider.NewDefaultCircuitBreaker("my-provider",
        provider)

listenerID := cb.AddListener(func(providerID string, oldState,
        newState llmprovider.CircuitState) {
    log.Printf("Provider %s: %s -> %s", providerID, oldState,
        newState)

    if newState == llmprovider.CircuitOpen {
        // Alert monitoring system
```

```
        alerting.NotifyProviderDown(providerID)
    }
})

// Later, remove the listener when no longer needed
cb.RemoveListener(listenerID)
```

Managing Multiple Providers

CircuitBreakerManager

```
// Create a manager with custom config
config := llmprovider.CircuitBreakerConfig{
    FailureThreshold:    5,
    SuccessThreshold:    2,
    Timeout:             30 * time.Second,
    HalfOpenMaxRequests: 3,
}
manager := llmprovider.NewCircuitBreakerManager(config)

// Register providers
cbClaude := manager.Register("claude", claudeProvider)
cbGemini := manager.Register("gemini", geminiProvider)
cbDeepSeek := manager.Register("deepseek", deepseekProvider)

// Get available providers (closed or half-open circuits)
available := manager.GetAvailableProviders()
// available might be: ["claude", "gemini", "deepseek"]

// Get a specific circuit breaker
cb, exists := manager.Get("claude")
if exists {
    resp, err := cb.Complete(ctx, req)
    // ...
}

// View all stats
stats := manager.GetAllStats()
for providerID, stat := range stats {
    fmt.Printf("%s: state=%s, failures=%d, requests=%d\n",
        providerID, stat.State, stat.TotalFailures,
        stat.TotalRequests)
}

// Reset all circuit breakers (e.g., after a network outage is
        resolved)
manager.ResetAll()
```

Using the Health Monitor

Basic Monitoring

```
// Create a health monitor with default settings (30s interval)
monitor := llmprovider.NewDefaultHealthMonitor()

// Register providers
monitor.RegisterProvider("claude", claudeProvider)
monitor.RegisterProvider("gemini", geminiProvider)
monitor.RegisterProvider("deepseek", deepseekProvider)

// Add a listener for health changes
monitor.AddListener(func(providerID string, oldStatus, newStatus
        llmprovider.HealthStatus) {
    log.Printf("Health change: %s %s -> %s", providerID,
        oldStatus, newStatus)
})

// Start monitoring
monitor.Start()
defer monitor.Stop()

// Query health status
health, ok := monitor.GetHealth("claude")
if ok {
    fmt.Printf("Claude: status=%s, latency=%v, checks=%d\n",
        health.Status, health.Latency, health.CheckCount)
}

// Get all healthy providers
healthy := monitor.GetHealthyProviders()
fmt.Printf("Healthy providers: %v\n", healthy)

// Get aggregate system health
agg := monitor.GetAggregateHealth()
fmt.Printf("System: %s (%d/%d healthy)\n",
    agg.OverallStatus, agg.HealthyProviders, agg.TotalProviders)
```

Custom Configuration

```
config := llmprovider.HealthMonitorConfig{
    CheckInterval:      10 * time.Second, // Check every 10
        seconds
    HealthyThreshold:   1,                // 1 success to mark
        healthy
    UnhealthyThreshold: 5,                // 5 failures to mark
        unhealthy
    Timeout:            5 * time.Second,  // 5 second check
        timeout
    Enabled:            true,
```

```
}
monitor := llmprovider.NewHealthMonitor(config)
```

Force an Immediate Check

```
// Force a check outside the normal interval
err := monitor.ForceCheck("claude")
if err != nil {
    log.Printf("Force check failed: %v", err)
}
```

Using Retry Logic

ExecuteWithRetry

```
import "digital.vasic.llmprovider"

config := llmprovider.DefaultRetryConfig()

result, err := llmprovider.ExecuteWithRetry(ctx, config, func()
        (*http.Response, error) {
    req, _ := http.NewRequestWithContext(ctx, "POST", apiURL,
        body)
    req.Header.Set("Authorization", "Bearer "+apiKey)
    return http.DefaultClient.Do(req)
})

if err != nil {
    log.Printf("All %d attempts failed: %v", result.Attempts, err)
    return err
}

defer result.Response.Body.Close()
fmt.Printf("Success after %d attempts (total delay: %v)\n",
    result.Attempts, result.TotalDelay)
```

RetryableHTTPClient

```
// Create a retryable HTTP client
retryClient := llmprovider.NewRetryableHTTPClient(
    &http.Client{Timeout: 60 * time.Second},
    llmprovider.RetryConfig{
        MaxRetries:   5,
        InitialDelay: 500 * time.Millisecond,
        MaxDelay:     15 * time.Second,
        Multiplier:   2.0,
        JitterFactor: 0.2,
    },
```

```
)

// Use it like a normal HTTP client
req, _ := http.NewRequestWithContext(ctx, "POST", url, body)
resp, err := retryClient.Do(ctx, req)
if err != nil {
    return err
}
defer resp.Body.Close()
```

Checking Retryable Conditions

```
// Check if an HTTP status code is retryable
if llmprovider.IsRetryableStatusCode(resp.StatusCode) {
    log.Printf("HTTP %d is retryable, will retry",
        resp.StatusCode)
}

// Check if an error is retryable
if llmprovider.IsRetryableError(err) {
    log.Printf("Error is retryable: %v", err)
}

// Calculate backoff for a specific attempt
backoff := llmprovider.CalculateBackoff(3,
        llmprovider.DefaultRetryConfig())
fmt.Printf("Attempt 3 backoff: %v\n", backoff)
```

Combining Components

Full Provider Setup with Circuit Breaker, Health, and Retry

```
func setupProvider(name, apiKey, baseURL string)
        (llmprovider.LLMProvider, error) {
    // 1. Create the base provider
    provider := myprovider.NewMyProvider(apiKey, baseURL,
        "default-model")

    // 2. Wrap with circuit breaker
    cb := llmprovider.NewCircuitBreaker(name, provider,
        llmprovider.CircuitBreakerConfig{
        FailureThreshold:    5,
        SuccessThreshold:    2,
        Timeout:             30 * time.Second,
        HalfOpenMaxRequests: 3,
    })

    // 3. Register with health monitor
```

```
    healthMonitor.RegisterProvider(name, cb)

    // 4. Add circuit breaker listener to update health on state
        changes
    cb.AddListener(func(providerID string, oldState, newState
        llmprovider.CircuitState) {
        if newState == llmprovider.CircuitOpen {
            healthMonitor.RecordFailure(providerID,
        fmt.Errorf("circuit opened"))
        } else if newState == llmprovider.CircuitClosed {
            healthMonitor.RecordSuccess(providerID)
        }
    })

    return cb, nil
}
```

This pattern is used throughout HelixAgent to provide fault-tolerant access to all 43 LLM providers.